Combined reference: LLM deployment, document retrieval, and transmission-line TikZ diagram

Test Document: GPU Pool and Corpus System Documentation

Eugen Starschich

September 18, 2026

Abstract

End-to-end account of how a local large language model is deployed on the shared GPU pool — how GPUs are borrowed, why the work happens inside a container, where files must live to survive, how the model runtime is installed into an image without package manager access, and how the resulting endpoint is reached from a remote machine — followed by *corpus*, the retrieval system that answers questions from our own documents over that endpoint. Plus: TikZ representation of transmission line model.

Contents

I GPU Pool and LLM Deployment **3**

1 Architecture **3**
1.1 The Shared GPU Pool 3

2 Storage Model **3**

3 Ollama Runtime **3**

4 Model and Endpoint **4**

5 Remote Access (Laptop → GPU Host) **4**

6 Pitfalls **4**

II Document Retrieval System (corpus) **5**

7 Document retrieval on the endpoint: *corpus* **5**
7.1 What it is 5
7.2 Where everything runs 5
7.3 Ingest: documents in, index out 5
7.4 Query: three model calls, not one 5
7.5 Model operations 6
7.6 Everyday commands 6

8 Known limits **6**

A AC Transmission Line Model **7**

B Diagram Description **8**

C GPU Pool Commands **8**

D Corpus Commands **8**

Part I
GPU Pool and LLM Deployment

1 Architecture

Two machines are involved: the developer VM (dev-eugen) and the GPU host (camimo). The pool claim lifecycle is explicit (claim → release), the container home is ephemeral (data must live on /shared), and the tunnel forwarding the endpoint must survive container recreation.

1.1 The Shared GPU Pool

The GPU pool is not a Kubernetes cluster. It is a single physical box (camimo) with four GPUs, plus a DRA (Dynamic Resource Allocation) script that hands one GPU at a time to developers on first-come-first-served basis. The claim lifecycle is explicit:

```
# --- pool ------------------------------------------------------------
gpu status              # see current claim holder
gpu claim 1             # claim GPU 1 (blocks forever until release)
gpu shell               # interactive shell inside the container
gpu shell /path/to/script.sh    # run command and attach
gpu release             # release GPU; container destroyed
```

Claims do not expire, by design. A forgotten claim blocks the pool indefinitely.

2 Storage Model

Only /shared spans both machines. The container home is the GPU host's, not the developer VM's.

@l X@ **Location**	**Purpose**
`/shared/`	Persistent data (model weights, scripts, logs)
`container home`	Ephemeral (deleted on release)
`/work/raw`	Dashboard uploads (dev-eugen only)

Table 1: Storage model: what survives and where.

3 Ollama Runtime

Install once, then the model and the container survive independently.

```
# --- inside the container --------------------------------------------
nvidia-smi                              # verify GPU is claimed
ps -eo pid,etime,cmd | grep "[b]in/ollama serve"
tail -f /shared/$USER/ollama-serve.log
```

The script /shared/$USER/start − ollama.shsurvivesrelease.Afteralaterclaim :

```
gpu claim 1
gpu shell /shared/$USER/start-ollama.sh
```

4 Model and Endpoint

```
# --- model ------------------------------------------------------
ollama list
ollama pull  gpt-oss:120b
ollama run   gpt-oss:120b --verbose "prompt"
ollama stop  gpt-oss:120b       # unload from VRAM, keep on disk

# --- endpoint ---------------------------------------------------
curl http://localhost:11435/api/tags
curl http://localhost:11435/api/generate \
  -d '{"model":"gpt-oss:120b","prompt":"...","stream":false}'
```

The local port (11435) differs from the default (11434) because the relay re-quotes arguments; nested quoting does not survive.

5 Remote Access (Laptop → GPU Host)

From the developer VM, reach the GPU host over the office LAN. The SSH tunnel forwards the model endpoint so the laptop can talk to it as if it were local.

```
ssh -f -N -L 127.0.0.1:11434:172.17.0.4:11434 camimo
```

Confirm the bridge address first, as it may differ between claims:

```
ssh camimo "docker inspect -f \
  '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' gpu-eugen"
```

6 Pitfalls

@¿p4.4cm X@ **Symptom** **Cause and remedy**

Claim never released Claims do not expire, by design. A forgotten claim blocks the pool indefinitely. Release explicitly.

`Text file busy` on a freshly staged binary NFS write-back lag on a new executable under `/shared`. It clears by itself; wait rather than re-copying.

`pgrep -f "ollama serve"` always matches The search pattern appears in the searching command's own command line. Use `grep "[b]in/ollama serve"`.

Quoted one-liner silently does nothing The relay re-quotes arguments; nested quoting does not survive. Put the commands in a script on `/shared`.

`curl` returns the wrong model A local Ollama already occupies port 11434, so the forward failed to bind and the query answered locally. Forward to a free local port such as 11435.

`ssh -N` appears to hang Expected. `-N` runs no remote command; it holds the tunnel open and prints nothing. Test from a second terminal.

Work lost on release Only `/shared` spans both machines. The container home is the GPU host's, not the developer VM's.

`<defunct>` processes accumulate Container PID 1 is `sleep infinity` and never reaps children. Harmless; cleared on release.

Table 2: Symptoms seen in practice, with cause and remedy.

Part II
Document Retrieval System (corpus)

7 Document retrieval on the endpoint: *corpus*

7.1 What it is

corpus is a retrieval-augmented question answering system over our own documents, running entirely on American Terawatt hardware and using the endpoint of Section 1 as its model. Documents are parsed once, split into passages, embedded, and indexed on local disk. At question time the relevant passages are retrieved and handed to `gpt-oss:120b` on the shared GPU pool, which answers from those passages and cites them. Nothing leaves the network, and nothing is sent to an external API.

The system deliberately does *not* fine-tune a model on the documents. Fine-tuning gives no reliable recall of specific facts, no citations, and would need re-running whenever a document changes. Retrieval at query time gives all three.

7.2 Where everything runs

Three machines are involved, and the split matters: the corpus lives next to the documents, the model lives on the GPU, and only questions and answers cross the network.

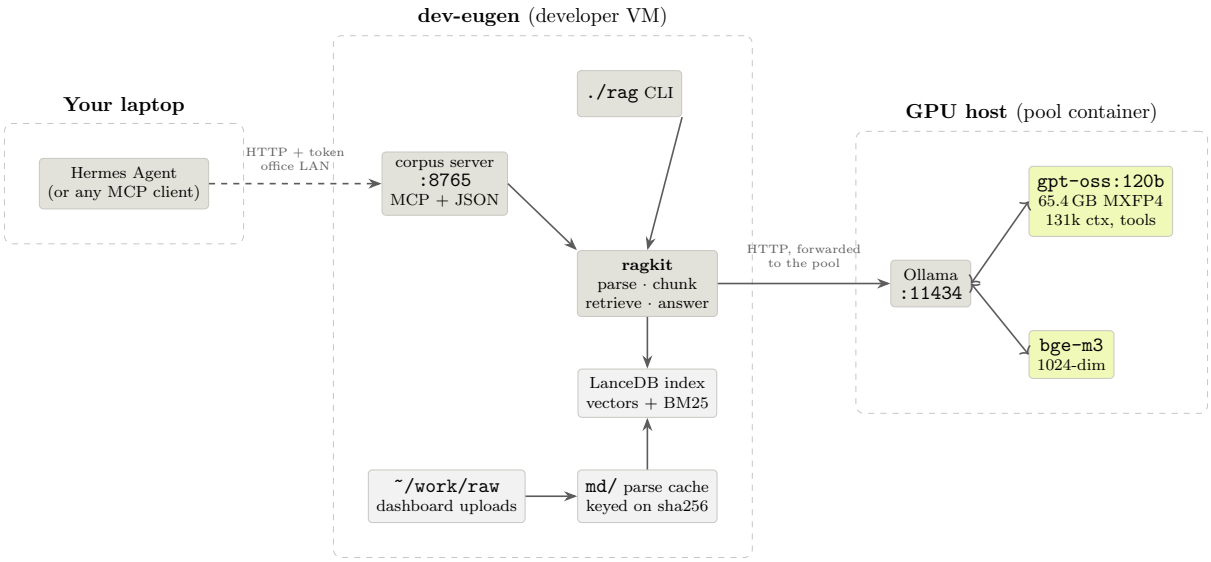


Figure 1: Where everything runs. Only questions and answers cross the network; the index stays beside the documents.

7.3 Ingest: documents in, index out

Ingest runs once per document, and is cached. Parsing is the slow step, so its output is stored in `md/` keyed on the source file's sha256 — re-running `./rag ingest` over a growing folder only touches files whose bytes changed.

7.4 Query: three model calls, not one

A question triggers an embedding call, then a rerank call, then the answer call. The database and the model never talk to each other; Python moves text between them.

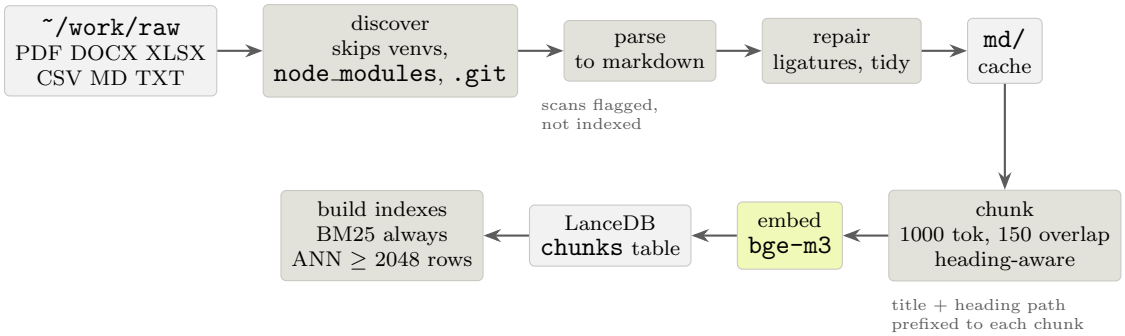


Figure 2: Ingest. Parsing is cached on the source's sha256, so re-running only touches documents whose bytes changed.

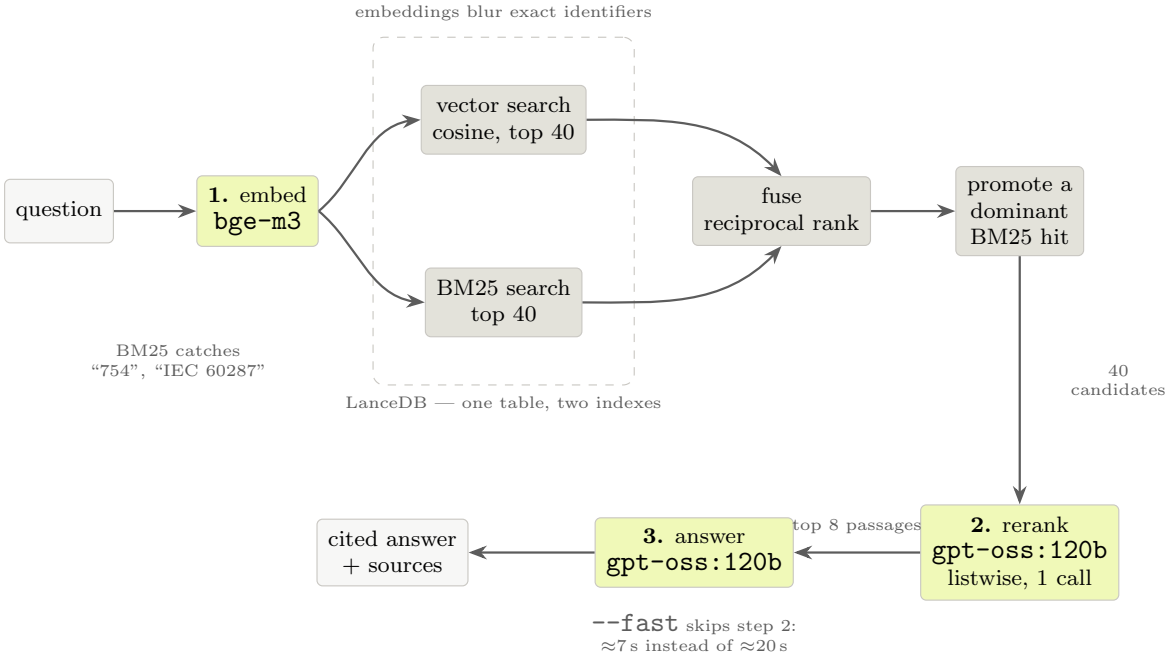


Figure 3: A query. Three model calls: embed the question, rerank the shortlist, then answer from the survivors.

7.5 Model operations

Four ways to interface with the corpus system:

- `./rag ask` - CLI, no server
- Python `import ragkit` - notebooks, own scripts
- MCP stdio - client spawns it
- MCP + JSON over HTTP `:8765` - Hermes and other agents

7.6 Everyday commands

8 Known limits

- **Scanned PDFs index nothing.** Ingest flags them rather than silently adding empty chunks. A vision model (`qwen2.5vl:32b`) is now on the pool and is the obvious OCR path

@l l l X@ **Interface Server? Auth Use it for**
`./rag ask` no — everyday use on the developer VM
`import ragkit` no — notebooks, own scripts
MCP stdio spawned — Claude Code on that box
MCP `/mcp` yes bearer Hermes and other agents
JSON `/ask`, `/search` yes bearer curl, scripts, any language

Table 3: The four interfaces and what each is for.

@l X@ **Task Command**
Index new uploads `./rag ingest`
Index elsewhere `./rag ingest /shared/eugen/brochures`
Re-parse everything `./rag ingest --force`
Rebuild from scratch `./rag ingest --reset`
Ask `./rag ask "..."`
Passages only `./rag search "..."`
What is indexed `./rag status`
Start the server `tmux new -s corpus-mcp`, then `./mcp-http --host 0.0.0.0`
Client details `./mcp-http --show-client-config`

Table 4: Everyday *corpus* commands on the developer VM.

if needed.

- **Damage already in the source is not repaired.** The eCIGRE brochure list contains titles that lost their ligatures before reaching the spreadsheet ("Model specication"), so a search for "specification" will not match them. Ligature repair runs only for PDFs, where the document's own text supplies a trustworthy vocabulary.
- **One writer.** LanceDB is embedded: concurrent readers are fine, two simultaneous ingests are not.
- **Rerank latency.** The 120B spends reasoning tokens before any visible output, so a reranked query costs ≈20 s.
- **LAN exposure.** With `--host 0.0.0.0` the bearer token is the only thing between the office network and every indexed document.

TikZ Transmission Line Diagram

A AC Transmission Line Model

The following TikZ diagram represents the transmission line model from image9.pdf:

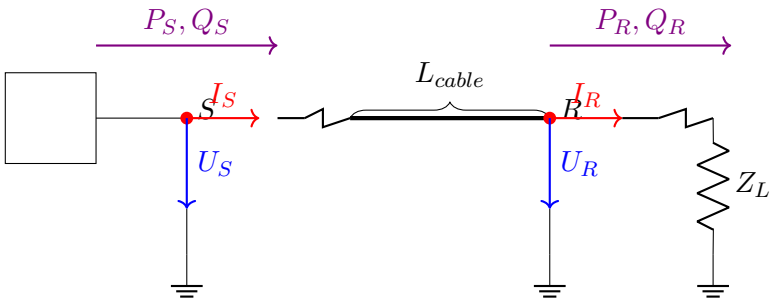


Figure 4: TikZ representation of the transmission line model from image9.pdf.

B Diagram Description

The diagram shows:

- Source box on the left with power flow labels (P_S, Q_S)
- Point S (sending end) marked with red dot
- Current I_S (red arrow) and voltage U_S (blue arrow)
- Cable segment L_{cable} with thick line and brace
- Point R (receiving end) marked with red dot
- Current I_R (red arrow) and voltage U_R (blue arrow)
- Power flow labels (P_R, Q_R) at receiving end
- Load Z_L (resistor) to ground

Command Reference

C GPU Pool Commands

```
# --- pool ------------------------------------------------------------
gpu status
gpu claim 1
gpu shell                       # interactive
gpu shell /path/to/script.sh    # one command
gpu release

# --- inside the container --------------------------------------------
nvidia-smi
ps -eo pid,etime,cmd | grep "[b]in/ollama serve"
tail -f /shared/$USER/ollama-serve.log

# --- model -----------------------------------------------------------
ollama list
ollama pull  gpt-oss:120b
ollama run   gpt-oss:120b --verbose "prompt"
ollama stop  gpt-oss:120b       # unload from VRAM, keep on disk

# --- endpoint --------------------------------------------------------
curl http://localhost:11435/api/tags
curl http://localhost:11435/api/generate \
  -d '{"model":"gpt-oss:120b","prompt":"...","stream":false}'
```

D Corpus Commands

```
# --- index -----------------------------------------------------------
./rag ingest                          # new uploads in ~/work/raw
./rag ingest /shared/eugen/brochures  # a folder elsewhere
./rag ingest --force                  # re-parse everything
./rag ingest --reset                  # rebuild from scratch
./rag status                          # what is indexed
```

```
# --- ask ------------------------------------------------------------
./rag ask    "..."                       # cited answer
./rag ask    "..." --fast                # skip the rerank call
./rag search "..."                       # passages only

# --- server ---------------------------------------------------------
tmux new -s corpus-mcp
./mcp-http --host 0.0.0.0                # MCP + JSON on :8765, bearer token
./mcp-http --show-client-config
```